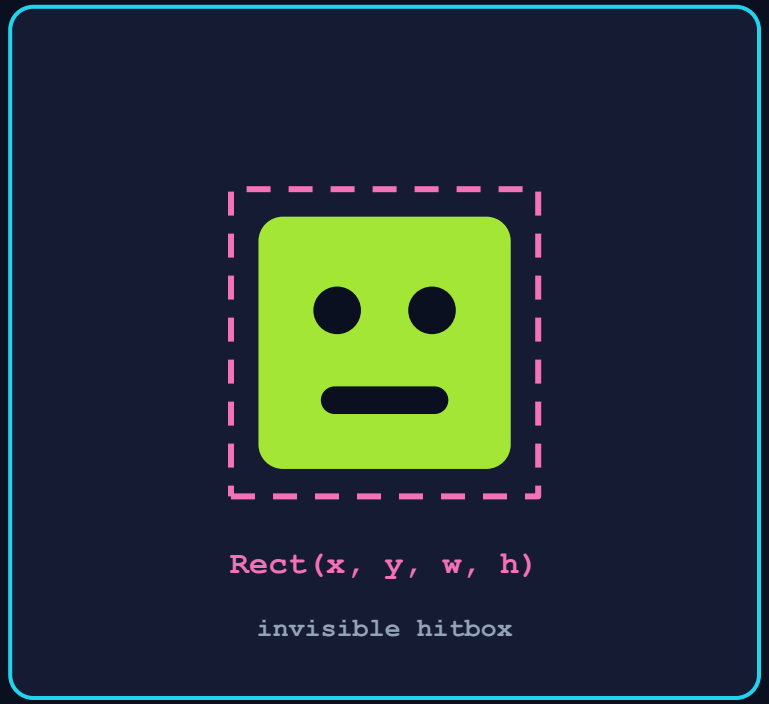
CCA GAME DEVELOPMENT

The Magic of Hitboxes (Rects)

Module 3 — Session 10

Teacher: Annisa Hendrawan

Junior High School — Global Prestasi School




```
player_rect = player_image.get_rect()
```

Today's Goal

By the end of this session, you will be able to:

- Understand what a pygame.Rect object is.
- Create an invisible hitbox for a game character.
- Set the position and size of a hitbox.
- Connect a hitbox to a sprite image.
- Use a Rect to track an object's location on the screen.

Today's Mission: Give your character an invisible hitbox!

Remember Session 9?

Last session, we learned how to:

- Load images using pygame.image.load().
- Display images using screen.blit().
- Replace basic shapes with character sprites.
- Add background images to our game.



How does Python know the exact position and size of our character?

Today, we will learn about `Rect`!

What Is a Hitbox?

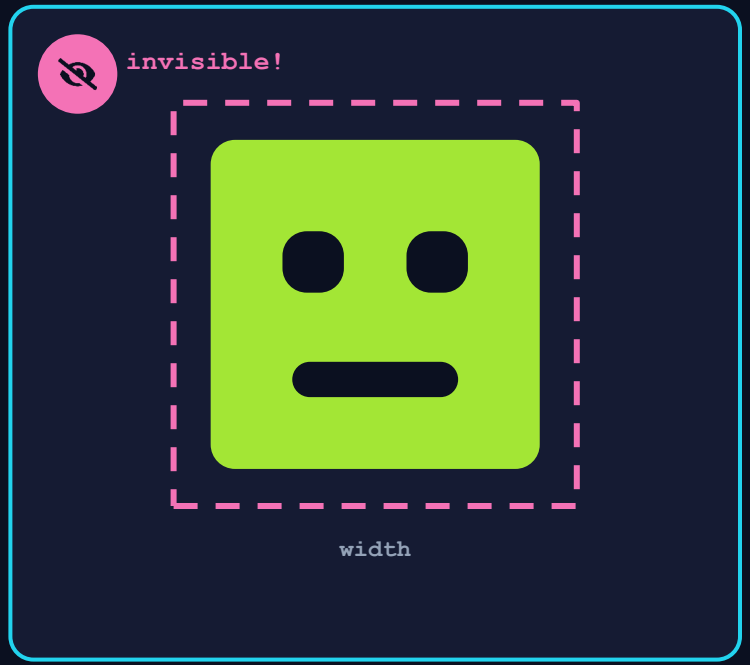
A Hitbox is an invisible area around a game object.

IT HELPS THE GAME TRACK

- Where the character is located.
- How wide the character is.
- How tall the character is.
- Where the object is on the screen.

Think of it like an invisible box surrounding your character!

Important: A hitbox does not have to be visible.



Meet pygame.Rect()

In Pygame, we can create a rectangle using:

```
pygame.Rect(x, y, width, height)
```

VALUE	MEANING
x	Horizontal position
y	Vertical position
width	Rectangle width
height	Rectangle height

```
player_rect = pygame.Rect(
  100, 200, 50, 50)
```

X position	100
Y position	200
Width	50
Height	50

Creating a Hitbox for Our Sprite

Remember our player image from Session 9? Create a Rect from its size.

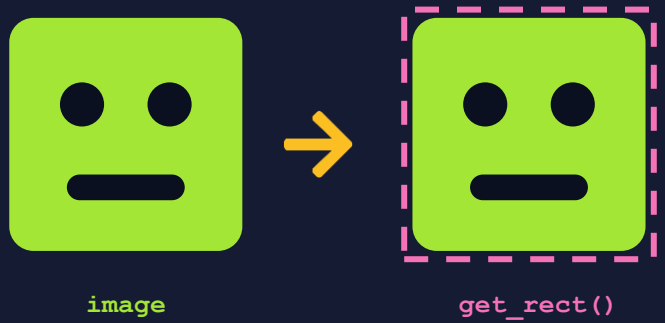
```
player_rect = player_image.get_rect()
```

`get_rect()` creates a rectangle.

`The rectangle` uses the image's width and height.

`player_rect` stores it.

We need to place the hitbox at the player's position!



Positioning the Hitbox

We can use .topleft to set the position of our Rect.

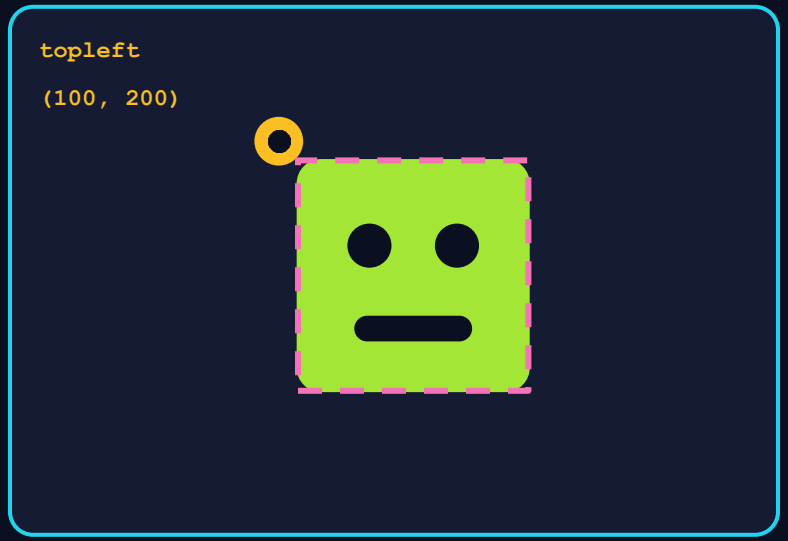
```
player_rect.topleft = (100, 200)
```

The left side starts at X = 100.

The top side starts at Y = 200.

Connect it to our existing player coordinates:

```
player_rect.topleft = (player_x, player_y)
```



Now the hitbox knows where our player is!

Moving the Hitbox

Previously, we moved the player with player_x:

```
player_x += player_speed
```

Now, we can move the Rect directly:

```
player_rect.x += player_speed
```



To move left:

```
player_rect.x -= player_speed
```

We can also access its vertical position:

```
player_rect.y += 5
```

New idea: A Rect has its own X and Y coordinates!

Drawing an Invisible Hitbox

Normally, we don't see the hitbox. While developing, we can draw it temporarily.

```
pygame.draw.rect(
    screen,
    (255, 0, 0),
    player_rect,
    2
)
```

`screen` = where to draw.

`(255, 0, 0)` = red color.

`player_rect` = the rectangle to draw.

`2` = outline thickness.



`red outline = hitbox`

Developer Tip: Remove or comment out this code when you no longer need it.

Let's Code!

Mini Mission: Give Your Character a Hitbox!

Starting with your Session 9 project:

1. Load your player image.
2. Create a Rect using player_image.get_rect().
3. Set the hitbox position using .topleft.
4. Move the Rect using .x or .y.
5. Display the player image using screen.blit().
6. Draw the hitbox temporarily to check its position.

Remember: The hitbox should follow the player!

VS CODE · PYTHON

Challenge Time!

Can You Customize Your Hitbox?

- Change the hitbox width.
- Change the hitbox height.
- Make the hitbox smaller than the character image.
- Make the hitbox wider than the character image.
- Try moving the hitbox independently.

Test → Observe → Explain

BONUS QUESTION

Why do you think games use invisible hitboxes instead of checking the image pixels directly?



What Did We Learn Today?

`pygame.Rect()` creates a rectangle.

`get_rect()` creates a Rect based on an image's dimensions.

`.topleft` sets the rectangle's position.

`.x and .y` change its position.

`A hitbox` helps the game track an object's location and size.

NEXT SESSION 11: COLLISION DETECTION

We will detect when two game objects touch each other using:

```
rect1.colliderect(rect2)
```

Get ready to make your game interactive!